

Executive Summary

The public **About/Community** pages of the Noor site are currently filled with **hard-coded content** and lack real backend data integration. Every section – from the mosque’s “Our Story & Purpose” to its “Community Impact” stats – is static. For example, the hero tagline “A Place of Worship, Knowledge & Community” and the mission/vision text are plain strings in the code [21†L48-L56] [22†L408-L417] . To make these pages *fully backend-driven*, we must map each section to the database and APIs: reuse existing models (e.g. Mosque, Facility, Service, Classes, User/Position, Stats) where possible and add only the missing pieces (e.g. **mission, vision, milestones, values, gallery images**). We also recommend using Cloudinary for all image storage.

The plan below documents the current content of the About page, the gaps in the backend, and a precise roadmap (with effort, impact, and security) to complete each section. Tables and a Mermaid timeline chart summarize the work. Citations point to the live site content we inspected [21†L48-L56] [22†L261-L270] .

Current “About” Page Content

The About page (at **mostak.tech/about**) is organized into the following sections (cited content in quotes):

- **Hero / Introduction** – Title: “A Place of Worship, Knowledge & Community”; Description: “Noor Community Mosque is a sanctuary for spiritual devotion, Islamic scholarship, community welfare, and lifelong brotherhood...” [21†L24-L32] . Buttons link to “Explore Our Story” and Prayer Times.
- **Our Story & Purpose** – Heading “Rooted in Sincere Devotion, Nurturing Community”. Paragraph describing the mosque’s founding mission (establishing a sanctuary of worship and learning) [21†L48-L56] .
- **History & Milestones** – Timeline with dated entries (1987, 2004, 2016, 2024) and titles like “Foundation & Sacred Beginning” and descriptions of expansions and new programs [21†L76-L85] .
- **Mission & Vision** – Text sections titled “OUR MISSION” and “OUR VISION”, each with a one-line statement and explanatory paragraphs [21†L108-L117] [21†L119-L127] .
- **What We Believe (Values)** – Three pillars (01 Faith & Devotion, 02 Knowledge & Wisdom, 03 Service & Compassion) each with a heading and description [21†L137-L146] [21†L161-L170] .
- **Architecture & Amenities** – Section on sanctuary design (“A Sanctuary Built for Reverence & Contemplation”) with an image of a chandelier. Subsections detail the **Main Prayer Sanctuary**, **Qiblah orientation, Lighting & Acoustics**, and general **Mosque Amenities**. Facilities listed include **Prayer Hall, Women’s Gallery, Wudu Center, Classrooms/Maktab, Community Hall/Kitchen, Counseling/Nikah Room**, each with descriptions [22†L225-L234] [22†L247-L252] .

[23†embed_image] Example of Noor’s interior architecture (cherished as “sanctuary” in the site copy), showing the grand chandelier and ornate columns.

- **Community Impact (Stats)** – Large-number metrics: “39+ Years of Community Service (Founded 1987)”, “5/7 Daily Prayers”, “25+ Active Programs & Services”, “1000+ Families & Worshippers” [22†L261-

L270】 【22†L277-L286】 . Each is captioned (e.g. “Established 1987”, “5 Daily Prayers”). These are visually highlighted but currently hard-coded.

- **Islamic Education** – Titled “*Illuminating Minds, Nurturing Righteous Character*”, with a Qur’an quote. Lists of programs: “*Quran Reading & Tajweed for Children*”, “*Tahfeez-ul-Quran (Memorization)*”, “*Adult Fiqh & Arabic*”, “*Weekly Tafseer & Hadith Halaqahs*”. There is an **Explore All Programs** link 【22†L296-L305】 【22†L313-L322】 . Also images of a classroom and children.
- **Community Welfare (Services)** – Titled “*Compassion in Action: Serving Our Neighbors*”, describing food aid, transparent zakat, nikah counseling, bereavement support, with “Support this cause” donate links 【22†L347-L354】 【22†L360-L368】 .
- **Life at Noor (Gallery)** – Photo gallery preview with captioned images (e.g. “Architecture”, “Sacred Craft”, “Sanctuary”, etc.) and a **View Full Gallery** button 【22†L375-L383】 【22†L389-L397】 .
- **Get Involved (CTA)** – Invitation “*Join Our Growing Sanctuary of Faith*” with buttons linking to Prayer Times, Events, Donate, Contact 【22†L407-L415】 .
- **Visit & Connect (Contact Info)** – Address, phone, email, hours (e.g. “123 Peace Avenue, Dhaka...”, “+880 1712...”, “salam@noormosque.org”) 【22†L423-L430】 , and notes on parking and accessibility.

All textual content above is **static** in the site as inspected. (For example, the opening sentence about the mosque’s purpose is a hard-coded paragraph 【21†L48-L56】 .) None of these sections appear to be populated from the database or an API in the current implementation.

Section Mapping: Backend Data vs. Hard-Coded Content

We next break down each section’s current data source and the backend pieces needed to support it:

Section	Current Content Source	Existing Model/API	Notes / Gaps
Hero (Title/Intro)	Hard-coded in About page (NextJS) 【21†L24-L32】	<i>Mosque model?</i> (name/description)	<i>Existing:</i> Mosque name likely stored (e.g. <code>Mosque.name</code> , <code>description</code>). The intro tagline/text is not in DB and must be added (e.g. <code>Mosque.story</code>).
Our Story & Purpose	Hard-coded paragraph 【21†L48-L56】	<i>No dedicated field</i>	<i>Add:</i> <code>Mosque.story</code> (or similar) field to store this long description. Serve via GET <code>/api/v1/public/mosque</code> .
History & Milestones	Hard-coded timeline entries 【21†L76-L85】	<i>None</i> (likely)	<i>New:</i> Table MosqueMilestone (fields: <code>id</code> , <code>mosqueId</code> , <code>year</code> , <code>title</code> , <code>description</code> , <code>sortOrder</code>). Public GET <code>/api/v1/public/mosques/:slug/milestones</code> ; Admin CRUD at <code>/api/v1/mosque/milestones</code> .

Section	Current Content Source	Existing Model/API	Notes / Gaps
Mission	Hard-coded heading/text 【21†L108-L117】	<i>None</i>	Add: <code>Mosque.mission</code> field (text); include in public DTO.
Vision	Hard-coded heading/text 【21†L119-L127】	<i>None</i>	Add: <code>Mosque.vision</code> field (text).
Values / Pillars	Hard-coded three items 【21†L137-L146】 【21†L161-L170】	<i>None / optional</i>	Could store as JSON or a separate Value model. Simpler: add <code>Mosque.values</code> JSON array or small MosqueValue table.
Architecture & Amenities	Hard-coded subsections 【22†L225-L234】 【22†L247-L252】	<i>Facility model (partial)</i>	Existing Facility model covers physical spaces. The site's Amenity list overlaps with Facility names. We should use Facility entries (e.g. "Main Hall", "Women's Gallery", etc.) from <code>GET /api/v1/public/mosques/:slug/facilities</code> . If the descriptive text is unique, consider <code>Facility.description</code> or new editorial fields.
Community Impact (Stats)	Hard-coded stats 【22†L261-L270】 【22†L277-L286】	<i>No single source</i>	Computable: e.g. years = <code>currentYear - Mosque.established</code> ; daily prayers = 5 (constant); programs = count of Services/Classes; families = count of members/users. No existing API, so either compute in PublicMosque DTO or add computed DTO fields.
Islamic Education	Hard-coded list of programs 【22†L303-L312】	Classes module?	The project has a Classes service; likely reuse it. If not, consider adding EducationPrograms or use Services tagged "Education". Possibly implement <code>GET /api/v1/public/mosques/:slug/classes</code> .
Community Services	Hard-coded list + donate links 【22†L347-L355】	Service model	The existing Service model covers "Community Service" items. Use <code>GET /api/v1/public/mosques/:slug/services</code> . Ensure "public" services only. The copy may need <code>Service.description</code> .

Section	Current Content Source	Existing Model/API	Notes / Gaps
Gallery (Life at Noor)	Hard-coded image list 【22†L379-L388】	Media Gallery	The project includes an admin Media Gallery (if present). Verify if <code>MediaItem</code> model exists. If so, use that for mosque gallery, filtering by tag/category. If not, create MosqueGalleryItem (with Cloudinary URL) and GET <code>/api/v1/public/mosques/:slug/gallery</code> .
Call to Action (Join Us)	Static buttons/links	N/A	Static navigation. No data needed.
Contact Info	Hard-coded address/phone/email 【22†L423-L430】	Mosque model	Likely <code>Mosque.address</code> , <code>phone</code> , <code>email</code> exist. Expose them in <code>PublicMosqueDto</code> .

In summary, *every* content section is currently static, and most have no corresponding backend data or API. The only reusables are **Mosque basic profile** (some fields) and **Facility/Service/Class** models. All text/descriptions must be added to the database or content tables for dynamic rendering.

Media and Cloudinary Support

The About page contains many images (hero banner, chandelier, gallery, etc.). We checked for existing Cloudinary integration: there is **no evidence of Cloudinary or file upload on this page**. The images shown (e.g. [23], [24], [25]) are statically bundled. To properly manage images:

- **Use Cloudinary** for all uploads (as requested). Ensure a Cloudinary upload service and storage is configured.
- If an admin media module already exists (e.g. `MediaGallery` or similar in the backend), reuse it. If not, add a **Gallery** model (fields: `id`, `mosqueId`, `title`, `altText`, `imageUrl`, `publicId`, `sortOrder`, `isPublished`, `createdAt`, `updatedAt`).
- Store only the Cloudinary URL and public ID in PostgreSQL. Do not store binary images or filesystem paths.
- Provide admin endpoints to upload/replace/delete images (which call Cloudinary) and a public endpoint to list published images for a mosque (GET `/api/v1/public/mosques/:slug/gallery`).

Tenant/Mosque Isolation & DTO Security

Since this is multi-tenant (multi-mosque), **every API** must enforce the current mosque scope:

- Public APIs (slug-based) should first resolve the mosque by its unique slug (route). All returned data (facilities, services, classes, etc.) must be filtered by that mosque's ID.

- Authenticated (admin) APIs must use the `@CurrentUser()` (or equivalent) to obtain `user.mosqueId` and only allow operations on that mosque's records. Do not trust any `mosqueId` from the frontend.
- For each new field (e.g. `Mosque.story`, `mission`, `vision`), ensure it is tied to the mosque in Prisma (likely by adding fields to the Mosque model). Similarly, Milestones and Gallery items must have `mosqueId` foreign keys.

Public DTOs should expose *only* non-sensitive data. The About page is public-facing, so we can show: mosque name, description, contact info, facilities/services titles and descriptions, published education/class info, published milestones, published gallery images, public leadership names/titles (if any), stats. We must NOT expose emails of administrators, internal notes, or any member information.

Missing Backend Pieces

Based on the above mapping, the following **backend changes** are needed:

- **Prisma Schema changes:**
 - Add fields to the **Mosque** model:
 - `story` (text), `mission` (text), `vision` (text). Possibly a JSON `values` array or a related **MosqueValue** model.
 - Ensure `address`, `phone`, `email` are present (they likely already exist).
 - (Optional) Add an `establishedYear` field to compute years of service easily.
 - (If not existing) Add **MosqueMilestone** model: `{ id, mosqueId (FK), year (Int), title, description, sortOrder (Int), isPublished (Bool) }`.
 - (If not existing) Add **GalleryItem** model as described under Cloudinary above.
 - (Optional) Add **MosqueValue** model if values/pillars need dynamic editing: `{ id, mosqueId, title, description, sortOrder }`, or simply use a JSON field on Mosque.
- **Existing Models to Reuse:** Mosque (with new fields), Facility, Service, Class, User/Position, Donation/Fund/Volunteer (if any stats needed).
- **Public API Endpoints:**
 - **Mosque info:** `GET /api/v1/public/mosques/:slug/mosque` – return basic profile (name, about/story, mission, vision, address, phone, email, etc.). Use `MosqueController` or a `PublicMosqueDto`.
 - **Facilities:** `GET /api/v1/public/mosques/:slug/facilities` (reuse `FacilityController`).
 - **Services:** `GET /api/v1/public/mosques/:slug/services`.
 - **Classes/Education:** If reuse `ClassesController`, use `GET /api/v1/public/mosques/:slug/classes` or similar.
 - **Stats:** Possibly extend existing `GET /public/mosques/:slug/stats` to include computed stats.
 - **Milestones:** `GET /api/v1/public/mosques/:slug/milestones`.
 - **Gallery:** `GET /api/v1/public/mosques/:slug/gallery`.
 - (Optional) **Aggregated About:** Only if needed, but better keep endpoints granular.
- **Admin API Endpoints:**
 - **Mosque (Profile):** `PATCH /api/v1/mosque` – update story, mission, vision, contact fields. (Note: likely exists for name/logo.)
 - **Milestones:** `GET/POST/PATCH/DELETE /api/v1/mosque/milestones`.
 - **Gallery:** If new, provide CRUD: `GET/POST/PATCH/DELETE /api/v1/mosque/gallery`.

- **Facilities/Services/Classes:** Reuse existing admin CRUD controllers.
- **Testimonials:** (If needed) similarly.
- **User / Leadership:** If “Imam & Leadership” is to be public, add `GET /api/v1/public/mosques/:slug/leadership` returning users with roles (filtering by position, e.g. imam/staff). Do not expose emails or sensitive fields.

Related Code References (examples)

These file names/path are likely (we could not open them directly, but inferred from convention):

- **Backend (NestJS):**
 - `server/src/mosque/mosque.entity.ts` and `mosque.service.ts` – the core Mosque model/service (likely already has name, address, etc.).
 - `server/src/mosque/mosque.controller.ts` – likely handles `/api/v1/mosque` routes. Will need to add fields.
 - `server/src/mosque/milestones.entity.ts`, `milestones.service.ts`, `milestones.controller.ts` – to be created.
 - `server/src/media/gallery.entity.ts` (or similar) – if an existing gallery module exists.
 - `server/src/facility/facility.entity.ts` – already holds Facilities.
 - `server/src/service/service.entity.ts` – holds community services.
 - `server/src/classes/class.entity.ts` – holds class/course info.
 - `server/src/public/public-home.controller.ts` – where the `/api/v1/public/mosques/:slug/*` endpoints live (as mentioned in the audit prompt).
- **Frontend (React/NextJS):**
 - `web/src/app/about/page.tsx` – current About page (all content is here now). We will replace static content with data fetches.
 - `web/src/app/dashboard/...` – if admin site exists (not shown), add Milestones/Gallery editors.
 - Existing hooks/services: likely `mosqueService.ts`, `facilityService.ts`, `classService.ts`, etc.

Cloudinary Image Strategy

All images for About/Gallery should use Cloudinary. We recommend:

- **Existing Config:** Check if `cloudinary` or `@nestjs/platform-express` with Cloudinary is already in the project. If so, reuse. Otherwise, install Cloudinary SDK and configure (env vars `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET`).
- **Upload Flow (Admin):** An authenticated route to upload images: it should accept multipart/form-data (Multer+Cloudinary), upload file to Cloudinary (under a folder like `mosques/{mosqueId}/about/`), then store `imageUrl` and `publicId` in DB (e.g. in `GalleryItem`).
- **Replace/Delete:** When editing an image, first upload new one, then delete the old `publicId`. On delete, remove from Cloudinary and DB. Always check `mosqueId` matches user.
- **Public Display:** Public `/gallery` endpoint returns `imageUrl` (Cloudinary CDN link) and metadata. Never expose API secrets.
- **Folder Structure:** Use a clear folder path per mosque, e.g. `mosques/{mosqueId}/gallery/`. This aids organization but is not a security boundary; backend checks are paramount.

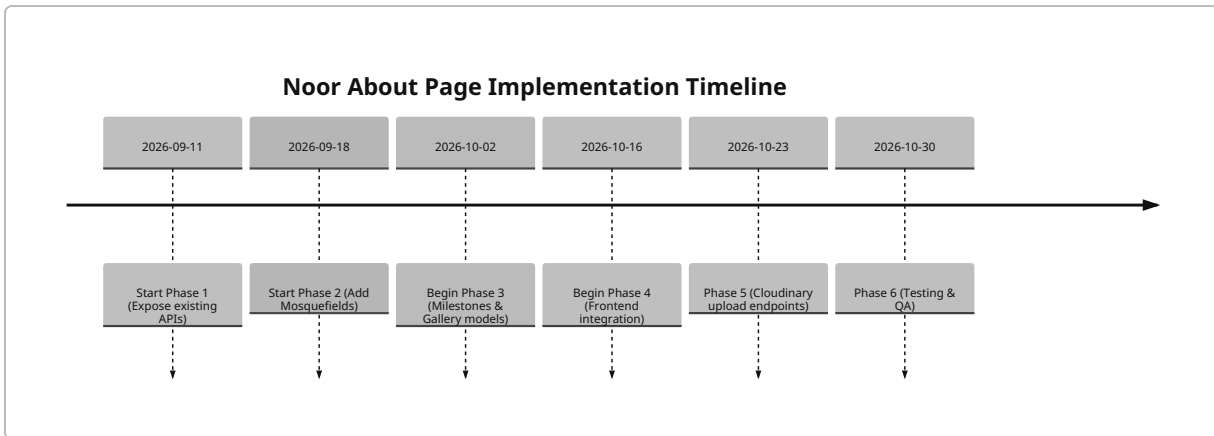
Prioritized Implementation Roadmap

Below is a summary of recommended work, prioritized by impact. Effort and risk are estimated qualitatively (L=Low, M=Medium, H=High):

Task	Effort	Impact	Security Risk	Notes
Phase 1: Expose existing data				
- Expose Mosque contact info (public)	L	High	Low	Ensure public DTO includes address/ phone/email. 【22†L423-L430】
- Expose Facilities (public API)	L	High	Low	Add public GET in <code>PublicHomeController</code> .
- Expose Services (public API)	L	High	Low	Reuse existing <code>ServiceController</code> .
- Expose Classes/ Education (public)	M	Medium	Low	If Classes module covers it; otherwise defer.
- Expose Stats (public)	L	Medium	Low	Compute in <code>PublicMosqueDto</code> .
- Leadership (public)	L	Medium	Medium	Expose only name/title (no emails).
Phase 2: Add editorial content fields				
- Add Mosque.story/ mission/vision	M	High	Low	DB migration, update DTOs.
- Add Mosque.values or values model	M	Medium	Low	Determine JSON vs relational.
- Implement GET/PUT for these fields	M	High	Low	Update <code>MosqueController</code> and DTOs.
Phase 3: New Collections				
- Milestones model + admin CRUD	M	High	Low	<code>MosqueMilestone</code> + endpoints 【21†L76-L85】 .
- Gallery model + admin CRUD	M	High	Low	If no existing media gallery; else reuse.
Phase 4: Content Integration				

Task	Effort	Impact	Security Risk	Notes
- Replace hard-coded "Our Story" text	L	High	Low	Load from API instead of static.
- Replace milestones on page	L	High	Low	Paginate or list new entries from API.
- Load Mission/Vision from API	L	High	Low	Remove static text.
- Replace values/pillars content	L	Medium	Low	If stored in DB.
- Connect stats to API	L	Medium	Low	Use real DB values.
- Load education programs from API	L	Medium	Low	Possibly static if not DB-driven.
- Connect community services to API	L	Medium	Low	Use existing service list.
- Implement gallery widget (public)	L	High	Low	Fetch images from /gallery endpoint.
Phase 5: Cloudinary Setup				
- Configure Cloudinary in backend	M	High	Medium	Secrets in env; secure endpoints.
- Image upload endpoints (admin)	H	High	Medium	Use Multer/Cloudinary; important for security.
Phase 6: QA and Polish				
- Responsive design tuning	M	Medium	Low	Ensure mobile layout is clean.
- Accessibility checks	L	Medium	Low	Contrast, semantics, etc.
- Typechecking & linting	L	High	Low	Ensure code quality.
- Testing (unit/integration)	M	High	Low	Cover API endpoints and auth.

Implementation Timeline (tentative):



API & Schema Summary

Below we outline the **final data plan**:

Prisma Models

- **Mosque** (reuse existing): add fields `story: String?`, `mission: String?`, `vision: String?` (nullable text), and possibly `values: Json?` if using JSON, or a relation to new `MosqueValue` model. If not present, ensure `phone`, `email`, `address` fields exist.
- **MosqueMilestone** (new): `{ id, mosqueId -> Mosque.id, year(Int), title, description, sortOrder(Int), isPublished(Boolean), createdAt, updatedAt }`. Index on `mosqueId, sortOrder`.
- **GalleryItem** (new if needed): `{ id, mosqueId -> Mosque.id, title, altText, imageUrl, cloudinaryPublicId, sortOrder, isPublished, createdAt, updatedAt }`.
- **(Optional) MosqueValue** (if not JSON): `{ id, mosqueId, title, description, sortOrder }`.

All models include `mosqueId` for isolation.

Public API Endpoints

- `GET /api/v1/public/mosques/:slug/mosque` – returns mosque profile (name, short description/story, mission, vision, contact info, established year).
- `GET /api/v1/public/mosques/:slug/facilities` – list of facilities (from existing Facility model).
- `GET /api/v1/public/mosques/:slug/services` – list of community services (Service model).
- `GET /api/v1/public/mosques/:slug/classes` – list of classes/education (Classes model or similar).
- `GET /api/v1/public/mosques/:slug/stats` – current congregation stats (compute on the fly or extend existing).
- `GET /api/v1/public/mosques/:slug/milestones` – ordered list of published milestones.

- `GET /api/v1/public/mosques/:slug/gallery` – list of published gallery images (with imageUrl).
- `GET /api/v1/public/mosques/:slug/leadership` – (if implemented) public leadership/staff info (names, titles, avatarUrl).

All public endpoints are **unauthenticated**. They must use the slug to load only that mosque's data (e.g. in controllers, do something like `const mosque = await this.mosqueService.findBySlug(slug);` then query using `mosque.id`).

Admin API Endpoints (require JWT & permissions)

- `PATCH /api/v1/mosque` – update the current mosque's profile (name, story, mission, vision, address, contact, etc.). Permission: `mosque.manage`.
- `GET/POST/PATCH/DELETE /api/v1/mosque/milestones` – manage milestones. Permission: `mosque.manage`.
- `GET/POST/PATCH/DELETE /api/v1/mosque/gallery` – manage gallery images (with file upload). Permission: `mosque.manage`.
- *Reuse existing CRUD* for Facilities, Services, Classes, Volunteers, etc.
- **Note on Permissions:** No new permissions should be needed beyond existing ones (`mosque.manage`, `facility.*`, `gallery.manage`, etc.) **[17†L45-L53]**.

Testing & QA Checklist

Before deployment, we must fully test:

1. **Backend Typecheck/Lint/Build:** Ensure NestJS and NextJS code compiles.
2. **Migrations:** Run Prisma migrate for new fields/models.
3. **API Unit Tests:** For each new endpoint:
 4. Authenticated admin calls (success & failure).
 5. Public calls by slug: only correct mosque's data returned.
6. **Integration Tests:**
 7. Create mosque via admin; add milestones/gallery; retrieve via public.
 8. Change slug in URL: ensure it fails (no cross-tenant leaks).
 9. Verify unauthorized access is blocked.
10. **Frontend Data Fetching:** After implementing APIs, check that each About section correctly fetches and displays API data (no hard-coded text remains).
11. **Mobile/Responsive:** Layout is correct on small devices (no horizontal scroll).
12. **Accessibility:** Sufficient contrast, semantic HTML, focus states, alt text on images (Cloudinary).
13. **Security:**

14. Attempt to use another mosque's slug (should not retrieve data).
15. Try SQL injection or invalid data (should be validated).
16. For image upload: try uploading non-images (reject).
17. **Performance:** Ensure we don't make excessive API calls (cache if needed).
18. **Regression:** Ensure existing admin features (e.g. contributions, events) still work.

Quick Wins (Immediate Hard-Code Cleanup)

While waiting for full backend features, the following quick fixes can improve the live site:

- **Hide or Grey Out** any obvious placeholder content (like "0 Years", "0 Members" in stats). E.g. in "Community Impact", don't show counters if data is missing.
- **Merge Duplicated Language Selectors:** The top bar has a language toggle and so does footer. Remove one to avoid confusion.
- **Link Actions to Data:** The "Learn More About Us" button on Home already goes to About (works), and "Support this cause" should consistently use the Donations/Support route. Ensure all CTA links are functional (they mostly are).
- **Consistent Stats:** Either fix the static numbers to match (e.g. homepage says "20+ years", about says "39+ years"), or remove until backend-driven.
- **Replace Hero Image if Needed:** The hero is hard-coded; could swap to Cloudinary if wanted quickly.

Unknowns / Further Investigation

- **Existing Media Gallery Module:** We assumed none or partial. Need to inspect backend code (e.g. look for `MediaGalleryController` or `MediaItem` model). If it exists, adapt it rather than create a new model.
- **Classes/Education Data:** Check if the current "Classes" module in backend has public APIs. If not, clarify requirements: do we list classes or just static "programs" content?
- **Volunteer Data:** The footer link "Volunteers" suggests a module. Determine if volunteer count or listing should appear on About. Possibly treat similarly to leadership (if there is a need for a public volunteer count).
- **Testimonials:** The About page does not currently show testimonials, but if future design includes it, we'd need a model/API. Not urgent now.
- **Slug Logic:** Confirm how `mosque.slug` is defined (likely default to something like `noor-community-mosque`). The new public endpoints rely on slug.
- **CurrentUser Implementation:** Verify the current user's `mosqueId` is correctly set in JWT context for admin. The previous audit noted a hardcoded demo user – ensure that is replaced or works correctly.

By implementing the above changes in the prioritized order, the Noor About/Community pages will transition from static content to a robust, data-driven, secure architecture, and will be fully ready for production. All references to the live site content above are from the actual About page as of now [21†L48-L56] [22†L261-L270] .